y for both GitLab.com and for self-managed users.

Goals

- Progressively rollout the new dependency of a PostgreSQL database instance for the registry for charts and omnibus deployments.
- Progressively rollout automation for the registry PostgreSQL database instance for charts and omnibus deployments.
- Develop processes and tools that self-managed admins can use to migrate existing registry deployments to the metadata database.
- Develop processes and tools that self-managed admins can use spin up fresh installs of the container registry which use the metadata database.
- Create a plan which will eventually allow us to fully drop support for original object storage metadata subsystem.

Non-Goals

- Developing new container registry features outside the scope of enabling admins to migrate to the metadata database.
- Determining lifecycle support decisions, such as when to default to the database, and when to end support for non-database registries.

Proposal

There are two main components that must be further developed in order for self-managed admins to move to the registry database: the deployment environment and the registry migration tooling.

For the deployment environments need to document what the user needs to do to set up their deployment such that the registry has access to a suitable database given the expected registry workload. As well as develop tooling and automation to ease the setup and maintenance of the registry database for new and existing deploys.

For the registry, we need to develop and validate import tooling which coordinates with the core import functionality which was used to migrate all container images on GitLab.com. Additionally, we should provide estimated import times for admins for each supported storage driver.

During the beta phase, we can highlight key features of our work to provide a quick reference for what features we have now, are planning, their statuses, and an excutive summary of the overall state of the migration experience.
This could be advertised to self-managed users via a simple chart, allowing them to tell at a glance the status of this project and determine if it is feature-complete enough for their needs and level of risk tolerance.

This should be documented in the container registry administration documentation, rather than in this blueprint. Providing this information there will place it in a familiar place for self-managed admins, will allow for logical cross-linking from other sections of the same document, such as from the garbage collection section.

For example:

The metadata database is in early beta for self-managed users. The core migration process for existing registries has been implemented, and online garbage collection is fully implemented. Certain database enabled features are only enabled for GitLab.com and automatic database provisioning for the registry database is not available.
See the table below for the status of features related to the container registry database.

Feature	Description	Status	Link
Import Tool	Allows existing deployments to migrate to the database.	Completed	Import Tool
Automatic Import Validation	Tests that the import maintained data integrity of imported images.	Backlog	Validate self-managed imports
Foo Bar	Lorem ipsum dolor sit amet.	Scheduled for 16.5	<LINK>

Structuring Support by Driver

The import operation heavily relies on the object storage driver implementation to iterate over all registry metadata so that it can be stored in the database. It's possible that implementation differences in the driver will make a meaningful impact on the performance and reliability of the import process.

The following two sections briefly summarize several points for and against structuring support by driver.

Arguments Opposed to Structuring Support by Driver

Each storage driver is well abstracted in the code, specifically the import process makes use of the following Methods:

- Walk
- List
- GetContent
- Stat
- Reader

Each of the methods is a read method we do not need to create or delete data via the object storage methods. Additionally, all of these methods are standard API methods.

Given that we're not mutating data via object storage as part of the import

process, we should not need to double-check these drivers or try to predict potential errors. Relying on user feedback during the beta to direct any efforts we should be making here could prevent us from scheduling unnecessary work.

Arguments in Favor of Structuring Support by Driver

Our experience with enhancing and supporting offline garbage collection has shown that while the storage driver implementation should not matter, it does. The drivers have proven to have important differences in performance and reliability. Many of the planned possible driver-related improvements are related to testing and stability, rather than outright new work for each driver.

In particular, retries and error reporting across storage drivers are not as standardized as one would hope for, and therefore there is a potential that a long-running import process could be interrupted by an error that could have been retried.

Creating import estimates based on combinations of the registry size and storage driver, would also be of use to self-managed admins, looking to schedule their migration. There will be a difference here between local filesystem storage and object storage and there could be a difference between the object storage providers as well.

Also, we could work with the importer to smooth out the differences in the storage drivers. Even without unified retryable error reporting from the storage drivers, we could have the importer retry more time and for more errors. There's a risk we would retry several times on non-retryable errors, but since no writes are being made to object storage, this should not ultimately be harmful.

Additionally, implementing Validate self-managed imports
would perform a consistency check against a sample of images before and after import which would lead to greater consistency across all storage driver implementations.

Design and Implementation Details

The Import Tool

The import tool
is a well-validated component of the container registry project that we have used from the beginning as a way to perform local testing. This tool is a thin wrapper over the core import functionality · the code which handles the import logic has been extensively validated.

While the core import functionality is solid, we must ensure that this tool and the surrounding process will enable non-expert users to import their registries with both minimal risk and with minimal support from GitLab team members. Therefore, the most important work remaining is crafting the UX of this tooling such that those goals are met. This
epic captures many of the
proposed improvements.

Design

The tool is designed such that a single execution flow can support both users with large registries with strict uptime requirements who can take advantage of a more involved process to reduce read-only time to the absolute minimum as well as users with small registries who benefit from a streamlined workflow. This is achieved via the same pre import, then full import cycle that was used on GitLab.com, along with an additional step to catalog all unreferenced blobs held in common storage.

One-Shot Import

In most cases, a user can simply choose to run the import tool while the registry is offline or read-only in mode. This will be similar to what admins must already do in order to run offline garbage collection. Each step completes in sequence, moving directly to the next. The command exits when the import process is complete and the registry is ready to make full use of the metadata database.

Minimal Downtime Import

For users with large registries and who are interested in the minimum possible downtime, each step can be ran independently when the tool is passed the appropriate flag. The user will first run the pre-import step while the registry is performing its usual workload. Once that has completed, and the user is ready to stop writes to the registry, the tag import step can be ran. As with the GitLab.com migration, importing tags requires that the registry be offline or in read-only mode. This step does the minimum possible work to achieve fast and efficient tag imports and will always be the fastest of the three steps, reducing the downtime component to a fraction of the total import time. The user can then bring up the registry configured to use the metadata database. After that, the user is free to run the third step during standard registry operations. This step makes any dangling blobs in common storage visible to the database and therefore the online garbage collection process.

Distribution Paths

Tooling, process, and documentation will need to be developed in order to support users who wish to use the metadata database, especially in regards to providing a foundation for the new database instance required for the migration.

For new deployments, we should wait until we've moved to general support, have automation in place for the registry database and migration, and have a major GitLab version bump before enabling the database by default for self-managed.

Omnibus

Charts

Alternative Solutions

Do Nothing

Pros

- The database and associated features are generally most useful for large-scale, high-availability deployments.
- Eliminate the need to support an additional logical or physical database for self-managed deployments.

Cons

- The registry on GitLab.com and the registry used by self-managed will greatly diverge in supported features over time.
- The maintenance burden of supporting two registry implementations will reduce the velocity at which new registry features can be released.
- The registry on GitLab.com stops being an effective way to validate changes before they are released to self-managed.
- Large self-managed users continue to not be able to scale the registry to suit their needs.

Gradual Migration

This approach would be to exactly replicate the GitLab.com migration on self-managed.

Pros

- Replicate an already successful process.
- Scope downtime by repository, rather than instance.

Cons

- Dramatically increased complexity in all aspects of the migration process.
- Greatly increased possibility of data consistency issues.
- Less clear demarcation of registry migration progress.

SECTION: engineering/architecture/design-documents/container_scanning_for_registry/_index.md

{{< engineering/design-document-header >}}

Summary

The Container Scanning for Registry feature enables the automatic execution of a container scanning job whenever a new image is pushed to the GitLab container registry. This feature helps in identifying vulnerabilities in container images early in the development process.

Motivation

Container Scanning scans Docker images when they are created during the pipeline. Images are

then stored in the GitLab Container Registry, and can be reused by other pipelines and deployment processes as stable images that will land in production.

Security status can change at any time even if there are no code changes, for example if an unknown vulnerability is disclosed to the public.

Developers and security teams need to know the current security status of the images stored in the GitLab Container Registry. When the advisory database is updated, they need to know when new vulnerabilities apply to existing images. Similarly, they also need to know when new vulnerabilities are introduced when a new image is pushed into the registry.

Goal

The primary goal of the Container Scanning for Registry feature is to enhance the security of containerized applications by automating the scanning process. This involves:

1. Automating the Detection Process: Ensuring that every new image pushed to the GitLab container registry is automatically scanned for vulnerabilities.
2. Early Identification of Vulnerabilities: Detecting and reporting vulnerabilities as soon as they are introduced.
3. Facilitate adoption of Container Scanning: Perform Container Scanning without requiring to configure a CI job in the .gitlab-ci.yml configuration file. A working runner is still required to execute the pipeline.
4. Providing Comprehensive View: Offering a detailed view of vulnerabilities that are accessible through a dedicated tab on the Vulnerability Report page.

Design and implementation details

How it's working

1. Trigger: When a new image is pushed to the GitLab container registry, a CI pipeline is automatically created with a container scanning job.
2. Report Generation: The reports generated by this job are marked with the report type containerscanningforregistry.
3. Vulnerability Reporting: The vulnerabilities identified by this job are displayed under Secure > Vulnerability Report page in Container Registry Vulnerabilities tab.

Implementation Details

Event Triggering

- Event Class: When a new image is pushed to the container registry, an event is fired using the class ContainerRegistryEvent.
  - Source Code: ContainerRegistryEvent

Image Pushed Event

- Condition: If the image tag is a supported tag, the necessary permissions and licenses are present, and the rate limit of 50 images scanned per project has not been exceeded, it publishes another event called ContainerRegistry::ImagePushedEvent.

- Source Code: ImagePushedEvent

CI Pipeline Creation

- Worker: The ImagePushedEvent triggers the ScanImageWorker, which schedules a CI pipeline via ScanImageService.
  - Source Code: ScanImageService

Container Scanning Tool Configuration

- Environment Variable: Based on the CI environment variable REGISTRYTRIGGERED: true, the container scanning tool sets the GitLab property name to containerscanningforregistry in the SBOM (Software Bill of Materials).
  - Source Code: SBOM Converter

Report Ingestion

- Property Check: During report ingestion, the system checks the property type and sets the sbomoccurrence.sourcetype to containerscanningforregistry.
  - Source Code: CycloneDX Properties Parser

Vulnerability Creation

- Report Type: During the creation of vulnerabilities, the vulnerabilityreporttype is set to containerscanningforregistry based on sbomoccurrence.sourcetype. This report type filter is used by the frontend to distinguish these vulnerabilities from other types.
  - Source Code: ContainerScanning FindingBuilder

Architecture Diagram

mermaid
```
graph TD;
    A[Image Push to GitLab Container Registry] --> B[ContainerRegistryEvent Trigger]
    B --> C[ContainerRegistry::ImagePushedEvent]
    C --> D[ScanImageWorker Starts]
    D --> E[ScanImageService Schedules CI Pipeline]
    E --> F[Container Scanning Job in CI Pipeline]
    F --> G[Set Property containerscanningforregistry in SBOM]
    G --> H[Report Ingestion Checks Property Type]
    H --> I[Set reporttype to containerscanningforregistry]
    I --> J[Vulnerabilities Displayed in Secure > Vulnerability Report]

    subgraph Container Registry
        A
    end

    subgraph GitLab CI/CD Pipeline
        E
        F
    end
```

```
    subgraph Vulnerability Management
        H
        I
        J
    end
```

This diagram captures the flow from the moment an image is pushed to the registry, through the event handling and CI pipeline, to the reporting and display of vulnerabilities. The various classes and services involved are highlighted at each step, providing a clear overview of the architecture.

Outline of the diagram

1. Image Push Event:
   - When a new image is pushed to the GitLab Container Registry.

2. ContainerRegistryEvent Trigger:
   - The ContainerRegistryEvent class fires an event.

3. ContainerRegistry::ImagePushedEvent:
   - If the image tag is supported and permissions are valid, it triggers ContainerRegistry::ImagePushedEvent.

4. ScanImageWorker:
   - ImagePushedEvent starts ScanImageWorker.

5. ScanImageService:
   - ScanImageWorker schedules a CI pipeline via ScanImageService.

6. Container Scanning Job:
   - CI pipeline includes a container scanning job.

7. SBOM Configuration:
   - Based on REGISTRYTRIGGERED: true, sets the property containerscanningforregistry.

8. Report Ingestion:
   - During ingestion, sets reporttype to containerscanningforregistry.

9. Vulnerability Report:
   - Vulnerabilities are displayed in the Secure > Vulnerability Report page under Container Registry Vulnerabilities tab.

SECTION: engineering/architecture/design-documents/covered_experience_slis/_index.md

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Glossary

- SLI: Service Level Indicator is a measure of the service level provided by a service provider to a customer.
- Application SLI: This is an SLI defined on the application side: the application decides what is ·good· for apdex and error portion. The SLI is associated with a service for monitoring in the runbooks repository. https://docs.gitlab.com/ee/development/applicationslis/
- Apdex (Application Performance Index): At GitLab in the context of covered experience SLIs, it is the completion of something within an acceptable amount of time, for example, the changes of a push are visible on the merge request within 30 seconds.
- User Journey: A comprehensive visualization or map that illustrates all the checkpoints, interactions, and emotions a customer experiences when engaging with a product, service, or brand, from initial awareness through purchase and beyond. In GitLab, it is the journey a user takes through the application. This can include multiple experiences. For example: Create a project -> Create an issue -> Create a merge request.
- Covered Experience: An action that a user takes inside the application that is covered with an indicator. A Covered Experience outlines the precise services, scenarios, and user interactions that establish clear performance expectations between a service provider and their client, some of which are covered by an SLI. Examples of experiences: ·create a project·, ·create an issue·, ·create a merge request·.
- Covered Experience SLI: An SLI implementation that represents an end-to-end flow of user interactions that may span multiple services.
- Multi-action Experience: A user journey that consists of multiple user interactions before completion, for example creating an issue consisting of 2 checkpoints: render new, submit form. We will not support this in the first iteration of Covered Experience SLIs.
- Single-action Experience: A user journey that consists of a single user interaction, for example ·view an issue· or ·add a comment to an issue·.
- Multi-service Experience: A journey that depends on multiple services to successfully complete, for example: a push gets received by GitLab-shell, which calls out to Rails, Gitaly and Sidekiq. A multi-service experience could be a single-action experience, only a single user-action is required for the experience, but it spans multiple services to be completed.
- Criteria: Each Experience can have one or more criteria that can be used to measure success. For example: ·The issue is successfully created· AND ·The issue is created fast enough·.
- Covered Experience Definition: The specification of the Covered Experience, distinguished by its coveredexperienceid. Example: coveredexperienceid="createmergerequest".
- Covered Experience Event: One instance of a Covered Experience. Distinguished by the properties coveredexperienceid and correlationid. Example: coveredexperienceid="createmergerequest" & correlationid="01G65Z755AFWAKHE12NY0CQ9FH".
- Covered Experience Checkpoint: This is the moment in the experience that we emit one event: the start of a request, the start of a job, the end of a request, the end of a job, etc.
We'll have at least one of these within a Covered Experience Event, but there can be multiple.

Motivation

While GitLab has robust service-level metrics through our SLI framework, we currently lack a systematic way to track and measure Covered Experiences that span multiple services. Our existing SLIs excel at measuring individual service performance but cannot effectively track the success/failure rate and performance of end-to-end user interactions. This gap makes it

challenging to:

- Understand the true user experience across service boundaries
- Set and monitor user-centric SLOs for complex user interactions
- Identify bottlenecks in multi-service flows
- Measure reliability and impact of incidents for customers and users

Goal

Track and measure Covered Experiences across GitLab services, establishing a framework for product teams to define and monitor Covered Experience SLIs.

How do Covered Experiences relate to User Journeys?

Covered Experiences are small interactions that users do on the platform. Covered Experiences focus specifically on single-actions users do that can be tracked and monitored through SLIs. While User Journeys represent comprehensive end-to-end paths a user might within the application. A User Journey can consist of many Covered Experiences, and a single Covered Experience can be part of many User Journeys.

mermaid
erDiagram
    coveredexperience ||--o{ coveredexperienceevent : "has many instances of"
    coveredexperience }o--o{ userjourney : "belongs to"
    userjourney ||--o{ userjourneyevent : "has many instances of"
    coveredexperienceevent ||--o{ userjourneyevent : "has many instances of"

Key relationships between the two concepts:

- Scope: A User Journey might encompass multiple Covered Experiences. For example, the User Journey of "contributing code to a project" might include several Covered Experiences like "git push," "merge request creation," and "CI pipeline execution".
- Measurability: Covered Experiences are specifically designed to be measurable through our SLI framework, with clear success criteria and thresholds. They should not include ambiguity through decisions that a user makes throughout their Journey.
- Implementation: User Journeys are often conceptual and used for product planning. Covered Experiences have specific technical implementations with instrumentation, metrics, and alerting.

Product defines the User Journeys and works together with Engineering to specify which Covered Experiences are part of those journeys. Here's a graphical representation:

graph src

Dos

- Create a framework for product teams to define important Covered Experience SLIs in a

structured way
- Develop an SDK that makes it easy for engineers to instrument Covered Experiences
- Support both GitLab.com and Dedicated deployments
- Enable measurement of Covered Experience success/failure rates and durations through metrics and logs
- Inform on the performance of Covered Experiences through SLIs that allow alerting on specified thresholds through our existing alerting framework

Don'ts

- Building a general-purpose distributed tracing solution
- Tracking client side timings, and time on the wire to clients. In the future, we want to add support for clients we build (IDE-extensions, our frontend), but we're keeping this out of scope in the first iteration.
- Real-time Covered Experience visualization or debugging tools
- Logs and metrics will be emitted from self-managed, but it won't officially support ingesting information from those instances as we don't have control over such environments

Unscoped

1. Other projects could benefit from Covered Experience SLIs, but are not part of the scope of this proposal. Such as:
    - Ensure critical user paths are well-tested and monitored (i.e. https://gitlab.com/groups/gitlab-org/quality/-/epics/144).
    The Covered Experience SLIs could provide data that can help identify end-to-end test coverage gaps for critical user paths.
    - Use Covered Experiences to inform Service Level Agreements (https://gitlab.com/gitlab-com/gl-infra/mstaff/-/issues/423)
2. Implementing a Covered Experience Tracker. There's a proposal for implementing a new service, as the likely
next step (covered in epic 1540), however, it is prone to change
as we progress on the implementation of Covered Experiences SDK, and discover its nuances and leverage points.

Scope

The implementation consists of the following components:

1. Covered Experience Definition Framework
2. LabKit SDK

The project work items are scoped in the epic 1539.

The Covered Experience Definition will provide an specification for each Covered Experience SLI,
and the SDK will be used to instrument the services to emit events (metrics and logs).

Example:

mermaid

```
sequenceDiagram
    participant User
    participant Web as Web Service
    participant Worker
    participant Event as Logs and Metrics

    User->>Web: Request
    activate Web

    Web->>Event: Checkpoint 1, SDK Emit Start
    Web->>Worker: Enqueue Job
    %% enqueued job will have all the Covered Experience relevant context,
    %% such as correlationid, for example
    Note over Web,Worker: Including event metadata
    Web-->>User: Response
    deactivate Web

    Note over Worker: Job wait in queue

    Note over Worker: Job starts
    activate Worker

    alt Success Case
        Worker->>Worker: End Covered Experience
        Worker->>Event: Checkpoint 2, SDK Emit Success
    else Failure Case
        Worker->>Worker: End Covered Experience
        Worker->>Event: Checkpoint 2, SDK Emit Failure
    end

    deactivate Worker
```

Covered Experience Definition

- YAML-based Covered Experience definition authored by product teams
- Support for specifying success criteria

The Covered Experience definition will contain the following fields:

Field	Type	Required	Description	Example
coveredexperienceid	string	Yes	Covered Experience identifier	"mergerequestcreation"
description	string	Yes	Human readable description	"User creates a merge request"
featurecategory	string	Yes	GitLab feature category	"sourcecodemanagement"

|
| urgency | string | Yes | How quickly a process needs to complete based on user expectations | "syncfast" |

Non-exhaustive list of urgencies to be supported:

Threshold	Description	Examples	Value
syncfast	A user is awaiting a synchronous response which needs to be returned before they can continue with their action	A full-page render	2s
syncslow	A user is awaiting a synchronous response which needs to be returned before they can continue with their action, but which the user may accept a slower response	Displaying a full-text search response while displaying an amusement animation	5s
asyncfast	An async process which may block a user from continuing with their user journey	MR diff update after git push	15s
asyncslow	An async process which will not block a user and will not be immediately noticed as being slow	Notification following an assignment	5m

As product teams implement more Covered Experiences, the list of urgencies will grow to accomodate different scenarios.

Examples:

coveredexperienceid	description	featurecategory	urgency
mergerequestcreation	User creates a merge request	sourcecodemanagement	"syncfast"
gitpush	User pushes commits to a repository	sourcecodemanagement	"asyncfast"

Given that Application SLIs are implemented in the Rails monolith at the moment, it will also function as a
registry initially to store the definitions for the Covered Experience SLIs.

SDK Requirements

- Implementation in LabKit
- DSL for sending Covered Experience events and checkpoints

The SDK will emit 1 event in every checkpoint (each interaction along the entire flow):

gitlabcoveredexperiencecheckpointtotal

| LABEL | EXAMPLE VALUE
METRIC	LOG

---|---|---|
| coveredexperienceid | securityscan
| yes | yes |
| correlationid | f93ae47de7f848343cf85511b47923ce
| no | yes |
| featurecategory | vulnerabilitymanagement
| yes | yes |
| checkpoint | start \| intermediate \| end
| yes | yes |
| checkpointcategory | e.g. authorize (impose limited cardinality)
| no | yes |
| type | web
| yes | yes |
| meta | { "relevant attributes": "tailored for the specific event" }
i.e. https://docs.gitlab.com/development/logging/logging-context-metadata-through-rails-or-grape-requests | no | yes |

And 2 more events, emitted at the end of the flow, to signify error and success:

gitlabcoveredexperiencetotal

| LABEL | EXAMPLE VALUE
METRIC	LOG

---|---|---|
| coveredexperienceid | securityscan
| yes | yes |
| correlationid | f93ae47de7f848343cf85511b47923ce
| no | yes |
| featurecategory | vulnerabilitymanagement
| yes | yes |
| error | true \| false
| yes | yes |
| type | sidekiq
| yes | yes |
| meta | { "relevant attributes": "tailored for the specific event" }
i.e. https://docs.gitlab.com/development/logging/logging-context-metadata-through-rails-or-grape-requests | no | yes |

gitlabcoveredexperienceapdextotal

| LABEL | EXAMPLE VALUE
METRIC	LOG

---------------------|--------|-----|
| coveredexperienceid | securityscan
| yes | yes |
| correlationid | f93ae47de7f848343cf85511b47923ce
| no | yes |
| featurecategory | vulnerabilitymanagement
| yes | yes |
| success | true \| false
| yes | yes |
| type | sidekiq
| yes | yes |
| meta | { "relevant attribute to the event": "tailored for the specific
event" }
 i.e. https://docs.gitlab.com/development/logging/logging-context-metadata-through-rails-or-grape-requests | no | yes |

Alternative Solutions

1. Do Nothing
   Pros:
   - No implementation cost

   Cons:
   - Continue lacking end-to-end covered experience visibility and measurement
   - Harder to set meaningful SLOs
   - Miss opportunities for better capturing perceived user experience and testing coverage

---
SECTION: engineering/architecture/design-documents/covered_experience_slis/next_step.md

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Motivation

Continuing the work from Covered Experience SLIs, this addendum focus on the augmentation of the Covered Experience Framework
by introducing a new service, the Covered Experience Tracker, that will add the capability of tracking Covered Experiences that
never complete (due to errors mid journey), or do not complete within its expected lifetime.

This is being kept separate from the main blueprint for the reason that it is subject to change. The content presented here is aspirational only.

Scope

The Covered Experience Tracker is going to be responsible for the Covered Experience time out verification --

especially relevant for tracking the asynchronous Covered Experience SLIs.

With the SDK consolidated, we can move and centralize the functionality of emitting events to this service, removing the complexity from the SDK.

The project will consist of:

1. Covered Experience Tracker
2. LabKit SDK

Example:

```
mermaid
flowchart LR
    User((User))

    subgraph ServiceA
        subgraph ProcessA
            LabKit
        end
    end
    subgraph ServiceB
        subgraph ProcessB
            LabKitB
        end
    end

    subgraph tracker[Tracker]
        missingend[Missing end event]
        timeoutcheck{Timeout check}
        timeoutaction[Timeout action]

        missingend --> timeoutcheck
        timeoutcheck --No timeout--> missingend
        timeoutcheck --Timeout reached--> timeoutaction
    end

    User --Request--> ServiceA
    LabKit --Checkpoint--> tracker
    ServiceA --> ServiceB
    LabKitB --Checkpoint--> tracker
```

The project work items are scoped in the epic 1540.

Covered Experience Tracker

- Centralized Covered Experience state tracking
- Sensible time to live (TTL) threshold for Covered Experience duration
- State is stored and managed by Redis. Allowing the querying of stale Covered Experiences,

timing out after configured threshold.
- Authentication
- Deployments:
  - Runway service for GitLab.com
  - Runway hosted service for Dedicated

It will serve an endpoint that will respond to the client generated payload:

Field	Type	Required	Description	Example
correlationid	string (ULID)	Yes	Unique identifier for the covered experience	"01JP0EM7HB39WSJNR4662MYZ6V"
coveredexperienceid	string	Yes	Identification of the Covered Experience	"httprequest"
checkpoint	string	Yes	Which step in the lifecycle	"start" | "end" | "intermediate"
checkpointcategory	string	No	A domain specific category for the checkpoint. TBD: impose limited cardinality.	"authorize"
type	string	Yes	Service/component generating the event	"web", "database"
featurecategory	string	Yes	GitLab feature category	"sourcecodemanagement"
urgency	string	Yes	How quickly a process needs to complete based on user expectations	"syncfast"
clienttimestamp	string (ISO-8601)	Yes	Timestamp when event occurred	"2025-02-06T14:30:00Z"
servertimestamp	string (ISO-8601)	No (Response only)	Server processing timestamp	"2025-02-06T14:30:00.123Z"
meta	object	No	i.e. https://docs.gitlab.com/development/logging/logging-context-metadata-through-rails-or-grape-requests	

A background process verifies all stale Covered Experiences and clears them out, emitting failure events.

Components interactions given a synchronous Covered Experience:

mermaid
sequenceDiagram
    participant User
    participant App as Service A
    participant AppB as Service B

```
    participant Tracker as Covered Experience Tracker
    participant Redis
    participant Event as Logs and Metrics

    User->>App: Request
    activate App

    App->>Tracker: Checkpoint 1
    Tracker->>Redis: Store Initial State
    Tracker->>Event: Emit Start Event
    Tracker-->>App: Response

    App->>AppB: Forward Request
    activate AppB

    AppB->>Tracker: Checkpoint 2
    Tracker->>Redis: Update State
    Tracker->>Event: Emit Intermediate Event
    Tracker-->>AppB: Response

    AppB-->>App: Response
    deactivate AppB

    App->>Tracker: Checkpoint 3
    Tracker->>Redis: Mark Complete
    Tracker->>Event: Emit Success Event
    Tracker-->>App: Response

    App-->>User: Response
    deactivate App

    loop Expired Covered Experience
        Tracker->>Redis: Check for Timeout
        alt Timeout Reached
            Tracker->>Redis: Mark Failed
            Tracker->>Event: Emit Failure Event
        end
    end
```

Components interactions given an asynchronous Covered Experience:

```
mermaid
sequenceDiagram
    participant User
    participant Web as Web Service
    participant Worker
    participant Tracker as Covered Experience Tracker
    participant Redis
    participant Event as Logs and Metrics
```

```
User->>Web: Request
activate Web

Web->>Tracker: Checkpoint 1
Tracker->>Redis: Store Initial State
Tracker->>Event: Emit Start Event
Tracker-->>Web: Response

Web->>Worker: Enqueue Job
Web-->>User: Response
deactivate Web

Note over Worker: Job wait in queue

Note over Worker: Job starts
activate Worker
Worker->>Tracker: Checkpoint 2
Tracker->>Redis: Update State

alt Success Case
    Worker->>Tracker: End Covered Experience (Success)
    Tracker->>Redis: Mark Complete
    Tracker->>Event: Emit Success Event
else Failure Case
    Worker->>Tracker: End Covered Experience (Failed)
    Tracker->>Redis: Mark Failed
    Tracker->>Event: Emit Failure Event
end

Tracker-->>Worker: Response
deactivate Worker

loop Expired Covered Experiences
    Tracker->>Redis: Check for Missing End Events
    alt Timeout Reached
        Tracker->>Redis: Mark Failed
        Tracker->>Event: Emit Failure Event
    end
end
```

Authentication

Authentication between the SDK and the Covered Experience Tracker is required to prevent the sending of unexpected events that could distort the reliability metrics of GitLab features.
E.g. the ai-gateway authentication and authorization.

SDK Requirements

- Trigger events towards the Tracker, instead of directly pushing events

- Automatic retries with exponential backoff for sending events to the Covered Experience Tracker

---
SECTION: engineering/architecture/design-documents/custom_models/_index.md

{{< engineering/design-document-header >}}

This Blueprint describes support for customer self-deployments of Mistral LLMs as a backend for GitLab Duo features, as an alternative to the default Vertex or Anthropic models offered on GitLab Dedicated and .com. This initiative supports both internet connected and air-gapped GitLab deployments.

Motivation

Self-hosted LLM models allow customers to manage the end-to-end transmission of requests to enterprise-hosted LLM backends for GitLab Duo features, and keep all requests within their enterprise network. GitLab provides as a default LLM backends of Google Vertex and Anthropic, hosted externally to GitLab. GitLab Duo feature developers are able to access other LLM choices via the AI Gateway. More details on model and region information can be found here.

Goals

Self-Managed models serve sophisticated customers capable of managing their own LLM infrastructure. GitLab provides the option to connect supported models to LLM features. Model-specific prompts and GitLab Duo feature support is provided by the self-hosted models feature.

- Choice of LLM models
- Ability to keep all data and request/response logs within their own domain
- Ability to select specific GitLab Duo Features for their users
- Non-reliance on the .com AI Gateway

Non-Goals

Other features that are goals of the Custom Models group and which may have some future overlap are explicitly out of scope for the current iteration of this blueprint. These include:

- Local Models
- RAG
- Fine Tuning
- GitLab managed hosting of open source models, other than the current supported third party models.

Proposal

GitLab will provide support for specific LLMs hosted in a customer's infrastructure. The customer will self-host the AI Gateway, and self-host one or more LLMs from a predefined list. Customers will then configure their GitLab instance for specific models by LLM feature. A different model can be chosen for each GitLab Duo feature.

This feature is accessible at the instance-level and is intended for use in GitLab Self-Managed instances.

Self-hosted model deployment is a GitLab Duo Enterprise Add-on.

Design and implementation details

Component Architecture

```
mermaid
graph LR
    a1 --> c1
    a2 --> b1
    b1 --> c1
    b3 --> b1
    b4 --> b1
    c1 --> c2
    c2 --> c3
    c3 --> d1
    d1 --> d2

    subgraph "User"
      a1[IDE Request]
      a2[Web / CLI Request]
    end

    subgraph "Self-Managed GitLab"
      b1[GitLab Duo Feature] <--> b2[Model & Feature-specific<br/>Prompt Retrieval]
      b3[GitLab Duo Feature<br/>Configuration]
      b4[LLM Serving Config]
    end

    subgraph "Self-Hosted AI Gateway"
      c1[Inbound API interface]
      c2[Model routing]
      c3[Model API interface]
    end

    subgraph "Self-Hosted LLM"
      d1[LoadBalancer]
      d2[GPU-based backend]
    end
```

Diagram Notes

- User request: A GitLab Duo Feature is accessed from one of three possible starting points (Web UI, IDE or Git CLI). The IDE communicates directly with the AI Gateway.
- LLM Serving Config: The existence of a customer-hosted model along with its connectivity information is declared in GitLab Rails and exposed to the AI Gateway with an API.

- GitLab Duo Feature Configuration: For each supported GitLab Duo feature, a user may select a supported model and the associated prompts are automatically loaded.
- Prompt Retrieval: GitLab Rails chooses and processes the correct prompt(s) based on the GitLab Duo Feature and model being used
- Model Routing: The AI Gateway routes the request to the correct external AI endpoint. The current default for GitLab Duo features is either Vertex or Anthropic. If a Self-Managed model is used, the AI Gateway must route to the correct customer-hosted model's endpoint. The customer-hosted model server details are the LLM Serving Config and retrieved from GitLab Rails as an API call. They may be cached in the AI Gateway.
- Model API interface: Each model serving has its own endpoint signature. The AI Gateway needs to be able to communicate using the right signature. We will support commonly supported model serving formats such as the OpenAI API spec.

Configuration

Configuration is set at the GitLab instance-level; for each GitLab Duo feature a drop-down list of options will be presented. The following options will be available:

- Self-hosted model 1
- Self-hosted model n
- Feature Inactive

In the initial implementation a single self-hosted Model will be supported, but this will be expanded to a number of GitLab-defined models.

AI Gateway Deployment

Customers will be required to deploy a local instance of the AI Gateway in their own infrastructure. The AI Gateway can be installed using:

- Docker
- Helm Chart

The AI Gateway container is published to the GitLab Container Registry and DockerHub on every GitLab Release.

Prompt Support

For each supported model and supported GitLab Duo feature, prompts will be developed and evaluated by GitLab. Prompts are hosted on the AI Gateway repository.

Supported LLMs

The list of supported LLMs are available in the documentation.

RAG / Duo Chat tools

Most of the tools available to Duo Chat behave the same for self-hosted models as they do in the GitLab-hosted AI Gateway architecture. Below are the expections:

Duo Documentation search

Duo documentation search performed through the GitLab-managed AI Gateway (cloud.gitlab.com) relies on VertexAI Search,
which is not available for air-gapped customers. As a replacement, only within the scope of self-hosted, air-gapped customers, an index of GitLab documentation has been provided within the self-hosted AI Gateway.

This index is an SQLite database that allows for full-text search. An index is generated for each GitLab version and saved into a generic package registry. The index that matches the customer's GitLab version is then downloaded by the self-hosted AI Gateway.

Using a local index does bring some limitations:

- BM25 search performs worse in the presence of typos, and the performance also depends on how the index was built
- Given that the indexed tokens depend on how the corpus was cleaned (stemming, tokenisation, punctuation), the same text cleaning steps need to be applied to the user query for it to properly match the indexes
- Local search diverges from other already implemented solutions, and creates a split between self-managed and GitLab-hosted instances of the AI Gateway.

Over time, we intend to replace this solution with a self-hosted Elasticsearch/OpenSearch alternative, but as of now, the percentage of self-hosted customers that have Elasticsearch enabled is low.

For further discussion, refer to the proof of concept.

Index creation

Index creation and implementation is being worked on as part of this epic

Evaluation

Evaluation of the local-search is being worked on as part of this epic.

LLM-hosting

Customers will self-manage LLM hosting. We provided limited documentation on how customers can host their own LLMs

GitLab Duo License Management

The Self-Managed GitLab Rails will self-issue a token (same process as for .com) that the local AI Gateway can verify, to guarantee that cross-service communication is secure. Details

System Architectures

At this time a single system architecture only is supported. See the Out of Scope section for discussion on alternatives.

Self-Managed GitLab with self-hosted AI Gateway

This system architecture supports both a internet-connected GitLab and AI Gateway, or can be run in an air-gapped environment. Customers install a self-managed AI Gateway within their own infrastructure. The long-term vision for such installations is via Runway, but until that is available a Docker-based install will be supported.

Self-Managed customers who deploy a self-managed AI Gateway will only be able to access self-hosted models at this time. Future work around Bring Your Own Key may change that in the future.

Development Environment

Some related work completed for the development environment include:

- Include AI Gateway in GDK
- Developer setup for self-hosted models
- Centralized Evaluation Framework

Out of scope

- It would be possible to support customer self-hosted models within a customer's infrastructure for dedicated or .com customers, but this is not within scope at this time.
- Support for models other than those listed in the Supported LLMs section above.
- Support for modified models.

Out of scope System Architectures

There are no plans to support these system architectures at this time, this could change if there was sufficient customer demand.

Self-Managed GitLab with .com AI Gateway

In this out-of-scope architecture a self-managed customer continues to use the .com hosted AI gateway, but points back to self-managed models.

.com GitLab with .com AI Gateway

In this out-of-scope architecture .com customers point to self-managed models. This topology might be desired if there were better quality of results for a given feature by a specific model, or if customers could improve response latency by using their own model-serving infrastructure.

GitLab Dedicated

Support will not be provided for Dedicated customers to use a self-hosted AI Gateway and self-hosted models. Dedicated customers who use GitLab Duo features can access them via the .com AI Gateway. If there is customer demand for self-managed models for Dedicated customers, this can be considered in the future.

SECTION: engineering/architecture/design-documents/custom_roles/_index.md

{{< engineering/design-document-header >}}

Overview

GitLab Custom Roles lets Ultimate subscribers create roles with specific permissions that fit their team's needs. Instead of using only the standard roles (Guest, Reporter, Developer, Maintainer, Owner), administrators can create new roles with exactly the permissions they want. This works for both project/group permissions and admin area access.
For example, an Ultimate customer could create an "Engineer" role based on the Guest standard role with read code and admin merge requests abilities, but without abilities like admin issues.

Each role can be customized by turning specific permissions on, making it easy to give team members just the access they need. Currently, custom abilities can only be added to standard roles, not subtracted.

Custom roles vs default roles

In GitLab 15.9 and earlier, GitLab only had default roles as a permission system. In this system, there are a few predefined roles that are statically assigned to certain abilities. These default roles are not customizable by customers.

With custom roles, the customers can decide which abilities they want to assign to certain user groups. For example:

- In the default role system, reading of vulnerabilities is limited to at least the Developer role.
- In the custom role system, a customer can assign this ability to a new custom role based on any default role.

Like default roles, custom roles are inherited within a group hierarchy. If a user has custom role for a group, that user will also have a custom role for any projects or subgroups within the group.

Terminology

- Custom Roles: User-facing feature name for customer-defined groups of permissions
- Member Roles: Backend/API term for custom roles, stored in the memberroles table
- Access Levels: Backend term for the default roles, stored as integers in the database
- Base Role: The default role that a custom role is based upon (can be nil for admin-related roles)
- Additive Permissions: Additional permissions granted on top of the base role's permissions

The terms "permission" and "ability" are often used interchangeably.

- "Ability" is an action a user can do. These map to Declarative Policy abilities and live in

Policy classes in ee/app/policies/.
- "Permission" is how we refer to an ability in user-facing documentation. The documentation of permissions is manually generated so there is not necessarily a 1:1 mapping of the permissions listed in documentation and the abilities defined in Policy classes.

Architecture

Permission Models

Regular Permissions

- Based on a default role
- Has one or more additive permissions
- Scoped to projects and/or groups
- Hierarchical Permission Inheritance:
  - A user with different custom roles at different levels receives all permissions
  - Example: If a user has guest+readcode in Group A and guest+readvulnerability in Project B (within Group A),
    they will effectively have guest role with both readcode and readvulnerability permissions in Project B
  - When going down the hierarchy, only custom roles with same or higher base role can be added

Admin Permissions

- Not tied to base access levels
- Specific permissions for admin area access

Database Structure

- Individual custom roles are stored in the memberroles table (MemberRole model). It includes individual permissions and a baseaccesslevel value.
- A memberroles record is associated with top-level groups (not subgroups) via the namespaceid foreign key for regular custom roles SaaS. Regular custom roles on self-managed instances and admin custom roles are not associated with groups.
- A Group or project membership (members record) is associated with a custom role via the memberroleid foreign key. Each member (group or project) can be associated with one custom role
- A Group or project membership can be associated with any custom role that is defined on the root-level group of the group or project.
- The baseaccesslevel must be a valid access level.

- The baseaccesslevel determines which abilities are included in the custom role. For example, if the baseaccesslevel is 10, the custom role will include any abilities that a default Guest role would receive, plus any additional abilities that are enabled by the memberroles record by setting an attribute, such as readcode, to true.

- Admin-related roles use Users::UserMemberRole model with its usermemberroles table
- Custom roles for admin permissions always have nil baseaccesslevel

Permission Loading Architecture

The Authz::CustomAbility class orchestrates permission checking:

- Determines the appropriate preloader based on resource type (Project, Group, Admin)
- Validates permission eligibility (feature flags, license, etc.)
- Memoizes permission results for performance

We use specialized preloaders to efficiently load custom permissions for different resource types:

Project Permissions

- Handled by Authz::Project class
- Uses UserMemberRolesInProjectsPreloader to load permissions
- Returns project-specific custom permissions for a given user

Group Permissions

- Handled by Authz::Group class
- Uses UserMemberRolesInGroupsPreloader to load permissions
- Returns group-specific custom permissions for a given user

Admin Permissions

- Handled by Authz::Admin class
- Uses UserMemberRolesForAdminPreloader to load admin-specific permissions
- Returns admin-level custom permissions for a user

Finder

MemberRoles::RolesFinder is a search service class that filters and sorts member roles based on parameters like parent, ID, and type. It takes care of searching regular roles while for admin roles we have Members::AdminRolesFinder

Documentation and Maintenance

Automated Documentation

- Documentation is automatically generated using Rake tasks:
  - bundle exec rake gitlab:customroles:compiledocs for custom abilities list
  - bundle exec rake gitlab:graphql:compiledocs for GraphQL documentation

- Generated documentation ensures consistency between code and docs

Adding New Custom Abilities

- Generated using ./ee/bin/custom-ability <ABILITYNAME>
- YAML configuration in ee/config/customabilities
- Schema validation and spec file generation
- Feature flag support with customability<name> pattern

- Policy updates in GroupPolicy and/or ProjectPolicy + specific entities if needed
- For detailed process on how to add custom permissions, please refer to the respective contributor documentation page.

Development Status

- Regular custom roles framework is implemented and actively being expanded with new permissions
- Admin custom roles are in early development stages, behind a feature flag

  - Currently available only for self-managed instances
  - Planned expansion to SaaS environments within few releases

References

- Custom Roles Blueprint
- Technical Discovery for Custom Permissions MVC
- Designs for Custom Permissions MVC

SECTION: engineering/architecture/design-documents/dashboard_layout_framework/_index.md

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

<!--
Don't add a h1 headline. It'll be added automatically from the title front matter attribute.

For long pages, consider creating a table of contents.
-->

Table of Contents

Summary

Dashboards are at the heart of how our customers interact with their data. It is the means by which they are able to understand their data
and use it to meet their business needs.

However, at GitLab, our dashboards have always been inherently feature-focused, without clear UX guidance or
common UI framework instructing how to build a dashboard. The result is an inconsistent user experience across GitLab for customers.
Additionally for development teams, there is no way to easily exchange or reuse
existing dashboards resulting in increased development time and maintenance cost.

~"group::platform insights" is leading the design, development, and implementation of the unified dashboard vision at GitLab.
This vision began with the Dashboards Working Group and in March 2023 culminated in a new Pajamas dashboards pattern that laid the groundwork for what a dashboard
is.

Since then, ~"group::platform insights" has developed an initial dashboards framework for the analytics feature space.
This was built off the initial work for Product Analytics.
The framework has been adopted by ~"group::optimize" for the Value Stream Dashboard and AI Impact Analytics,
as well as currently being developed for our Security Dashboards.

The next stage of this work is to solidify the foundations of the dashboards framework, align on the UI/UX,
and what features the dashboards framework will support. There must be clear guidance on:

- What a dashboard is and isn't, along with what functionality a dashboard should provide.
- How to use the dashboards framework within existing features and new features.
- What guidelines should be followed when using the dashboards framework.
- How to contribute to the dashboards framework and the wider dashboards UI/UX.

Motivation

As part of the Data Unification and Insights effort, we are looking to unify and standardize our data offering
at GitLab. A core part of this work is aligning our UI/UX to how customers interact with their data and
gain insights on what they can do to meet their business needs. Implementing and adopting a standardized
dashboards framework will go a long way to meeting this need, whilst also giving us the foundation to
augment our existing offering with clearer visuals and AI integration.

Goals

- Clear guidelines on what constitutes a dashboard layout , what functionality it contains, and

how to use the framework.
- Outline a clear migration path for existing dashboards.
- Adopt the dashboards framework across GitLab, especially where data is being used for analysis.
- An agnostic dashboards framework, not tied to any one feature, giving engineers the tools needed to quickly and efficiently set up and use dashboards.
- Link uses of the dashboards framework together in preparation for dashboards navigation restructuring.

Non-Goals

- ~"group::platform insights" will not implement every possible piece of functionality, data source, visualization type, or dashboard.
  These will be driven by feature teams, with support from ~"group::platform insights".
- The dashboard layout framework does not include data exploration outside defined panel visualizations.
- The dashboard layout framework does not include user-driven customization of dashboards, only the building blocks of the dashboards themselves.
- The dashboard layout framework does not define where the dashboard should be placed in the navigation.

Proposal

With the above goals and motivation in mind, we want to outline a dashboard layout framework that provides the core
functionality, UI, and UX needed to efficiently develop a dashboard within GitLab that adheres to our Pajamas guidelines.
The structure outlined below describes what this will include, and how they will function.

Design and implementation details

The grid

In line with our design definition of a Grid, we require a cross-browser system
of rows and columns to snap panels into position. Panels must be able to be resized and repositioned in a deterministic and user friendly way.

The grid itself will support 12 columns, with an unlimited number of rows. Grid panels can be up to 12 columns wide,
allowing 1·12 panels per row. The height of a panel can be between 1 row and unlimited, enabling it to span any number
of rows as needed.

Each panel, within the grid, will have a minimum height of 125px. This minimum height gives space for padding, the title, and basic content.

At the medium breakpoint, the grid must collapse down to a singular
column. This will move every panel to a fully vertical